

AgentsGate

初級者向け導入ガイド

日本語版 — 英語版は *agentsgate-beginner-guide.pdf*

Version 1.3 ・ 2026-07-31 ・ for AgentsGate 0.1.3

目次 / Contents

1. AgentsGate とは？
2. インストール前の準備
3. インストール手順
4. 起動と基本動作確認
5. Claude Desktop との連携
6. データベース MCP サーバー（PostgreSQL / MySQL）連携
7. ダッシュボードの使い方
8. 実践シナリオ: PostgreSQL / MySQL でデータベース操作を保護する
9. テスト実行とデバッグ情報の確認
10. よく使うコマンド一覧
11. よくある質問・トラブルシューティング
12. アンインストール

1. AgentsGate とは？

AgentsGate は、あなたの AI エージェント（Claude、ChatGPT など）と、AI が使うツール（ファイル操作、データベース操作、コマンド実行、API 呼び出しなど）の間に置くセキュリティプロキシです。

AI エージェントが行うすべての操作は、まず AgentsGate を通過します。AgentsGate は各操作を記録し、リスクスコアを算出し、危険な操作を自動でブロックまたは承認待ちにします。

AgentsGate の役割（図）



AgentsGate は 2 段階で判定します。あらゆる操作を一括で扱う保護レベルと、個別の例外を扱うポリシールールです。

保護レベル

スコアだけでは「何をすべきか」を決められません。DROP TABLE は 1.00、SELECT * FROM users は 0.60 ですが、この 2 つは程度ではなく種類が違います。SELECT を通すためにしきい値を上げると、DELETE FROM orders (0.90) まで通ってしまいます。そのため組み込みルールにはカテゴリが付いており、保護レベルがカテゴリごとの扱いを決めます。

agentsgate level — いま何が止まるのか、その理由を表示します

agentsgate level strict — 変更します。ダッシュボードのヘッダにも同じ切り替えがあり、そちらで変更すると実行中のプロキシに即座に反映されます。

minimal — まったく取り返しがつかないものだけを止めます（**DROP TABLE**、**TRUNCATE**、**WHERE** 句のない **DELETE**、**mkfs**、デバイスへの **dd**）。使い捨てのプロジェクト向けです。

balanced — 既定。上記に加えて、認証情報ファイル（**.env**、**.pem**、**.ssh**）への書き込みを拒否し、まとめ削除（ディレクトリ削除、**rm -rf**、ワイルドカード）は確認を求めます。コードを書く、テストを走らせる、行を更新する、ファイルを 1 個削除する——いずれも確認なしで通ります。

strict — 個人情報の読み出し、外部への送信、シェル実行、ファイル 1 個の削除を承認対象に加え、まとめ削除は拒否します。自分だけのものではないデータを扱う場合向けです。

balanced は利便性のための設定であって、データ保護の姿勢ではありません。シェルコマンドは実行され、**users** という名前のテーブルも読み取れます。

AgentsGate が記録するもの、元に戻せるもの

AgentsGate は、ツールの種類を問わずエージェントのツール呼び出しをすべて記録します。ただし「元に戻せるか」は別の話で、その操作が何に触れたかによります。

ローカルファイル — 完全に復元できます。スコア 0.30 以上の操作の前に、対象ファイルがスナップショットとして保存されます。**agentsgate rollback <checkpointId>** で元の状態に戻せます。これから作成しようとしていたファイルには戻す中身が無いため、すでに存在するファイルだけが対象です。

データベース（**SQLite**、**PostgreSQL**、**MySQL**） — 行単位で復元できます。**INSERT**・**UPDATE**・**DELETE** や **DDL** の実行前に、保護付き MCP サーバーが対象テーブルを複製します。対象テーブルは **SQL** から自動的に判別されるので、指定は不要です。**agentsgate rollback** でその複製からテーブルを復元します。保持されている内容は **agentsgate snapshot list** で確認できます。

シェルコマンド — 記録はされますが、復元はできません。コマンドはリスクスコアと判定とともにログに残りますが、それが何をしたかは記録されていません。**AgentsGate** から見えるのはコマンド文字列だけで、そのコマンドが触れたファイルは分からないため、戻すためのスナップショットが存在しません。シェル経由のまとめ削除が **balanced** で承認待ちになるのはこのためです。後から戻せない以上、実行前に止めることが唯一の防御手段になります。

外部サービス（**Slack**、**Gmail**、**Google カレンダー**） — 部分的です。送信は取り消せません。**AgentsGate** にできるのは事前に止めることで、そのために **outbound** のカテゴリがあります。

ロールバックの仕組み

ファイルについては、**AgentsGate** は **~/.agentsgate/shadow-repo** に専用の **git** リポジトリ（シャドウリポジトリ）を持っています。これはプロジェクト自身の **git** 履歴とは無関係で、どこにも **push** されません。ローカルにのみ存在し、スナップショットを保持するためだけのものです。

危険な操作の前に、対象ファイルがこのリポジトリにコピーされてコミットされます。このコミットがチェックポイントです。ロールバックはそのコミットをチェックアウトし直すだけなので、復元は正確で、**git checkout** と同じ速さで完了します。

git であるため、同じファイルの世代は丸ごとの複製ではなく差分として保存されます。大きなファイルの履歴を長く保持しても容量はあまり増えません。

agentsgate checkpoints — 復元できる地点の一覧

agentsgate rollback <id> — その地点に復元

データベースはシャドウリポジトリには入りません。テーブルの内容はデータベース MCP サーバーが、データベースファイルの隣にある専用のスナップショット領域へ複製します。行を戻すこととファイルを戻すことは別の操作だからです。

ポリシールールによる個別調整

レベルは大雑把な設定です。3つの選択肢で、あらゆる種類の操作を一括で扱います。どれかが「ほぼ合っているが一部だけ違う」ときは、ポリシーファイルで個別の事例だけを調整できます。

ポリシールールは両方向に働きます。レベルが許可しているものを拒否する（厳しくする）ことも、レベルが止めるものを許可する（緩くする）こともできます。ポリシーはレベルの後に適用され、どちらの方向でもレベルより優先されます。そのため「全体的に厳しすぎるレベル」と「全体的に緩すぎるレベル」の二者択一を迫られることはありません。

厳しくする例：balanced は通常書き込みを許可しますが、/etc 配下への書き込みだけを拒否します。

```
{ "rules": [ { "id": "PROTECT_ETC", "match": { "pathPattern": "^/etc/" }, "action": "block" } ] }
```

緩くする例：balanced では承認待ちになる `rm -rf node_modules` を許可します。

```
{ "rules": [ { "id": "ALLOW_NODE_MODULES", "priority": 10, "match": { "tool": "/shell/i", "paramsMatch": { "command": "/rm -rf node_modules/" } }, "action": "allow" } ] }
```

ルールでも迂回できないものがあります。セッションの失効、クォータ超過、サーキットブレーカーの作動、レート制限——これらはスコア算出より前に操作を拒否するため、ルールは参照されません。

agentsgate policy list — 適用中のルール一覧

agentsgate policy test --tool=... --method=... — 信用する前に試す

詳細は docs/policy-guide.md にあります。

スコア範囲	判定	動作
0.0 ~ 0.29	allow（許可）	即時実行される
0.30 ~ 0.69	require_approval（承認待ち）	一時停止・チェックポイント保存・通知
0.70 ~ 1.0	block（ブロック）	拒否・理由をログに記録

2. インストール前の準備

AgentsGate をインストールする前に、以下のソフトウェアが必要です。

必要なもの	最低バージョン	確認コマンド
Node.js	20.0.0 以上	node --version
npm	8.0.0 以上	npm --version
Git	最近のバージョン	git --version

Node.js のインストール

Node.js がインストールされていない場合は、公式サイト <https://nodejs.org> から LTS バージョンをダウンロードしてください。

インストール後、ターミナルで確認します：

```
node --version    # v20.x.x 以上が表示されるはず
npm --version     # 8.x.x 以上が表示されるはず
```

3. インストール手順

npm でグローバルにインストールします（どこからでも agentsgate コマンドが使えるようになります）：

```
npm install -g agentsgate
```

インストールの確認：

```
agentsgate --version
```

AgentsGate v0.1.0 のような出力が表示されれば成功です。

💡 ヒント (macOS / Linux) : 「permission denied」エラーが出た場合は、コマンドの先頭に *sudo* を付けてください。

```
sudo npm install -g agentsgate
```

💡 ヒント (Windows) : 「agentsgate が認識されない」場合は、管理者権限でコマンドプロンプトを開いてから実行してください。

または *npm bin -g* で表示されるパスを *PATH* に追加してください。

4. 起動と基本動作確認

デフォルト設定で AgentsGate を起動します：

```
agentsgate start
```

起動すると以下のような出力が表示されます：

```
AgentsGate v0.1.0 started
Proxy:      http://localhost:4000
Dashboard:  http://localhost:4001
Database:   ~/.agentsgate/agentsgate.db
Run `agentsgate stop` to stop.
```

AgentsGate はバックグラウンドで動作を続けます。ターミナルを閉じて構いません。停止するには `agentsgate stop` を実行してください。（前面で実行したい場合は `agentsgate start --foreground`）

起動確認

別のターミナルウィンドウを開き、以下を実行します：

```
agentsgate status
```

プロキシがアクティブであることを示すステータスが表示されれば正常です。

カスタムポートで起動する場合

ポート 4000 または 4001 が他のアプリで使われている場合：

```
agentsgate start 4100
```

5. Claude Desktop との連携

Claude Desktop を使っている場合、AgentsGate が自動的に設定を行います。これにより、Claude のすべてのツール呼び出しが自動的に AgentsGate を通過します。

⚠ このコマンドを実行する前に *Claude Desktop* を閉じてください。

```
agentsgate inject
```

成功すると以下が表示されます：

```
Injected AgentsGate proxy into 1 server(s):
+ filesystem

Config:  ~/Library/Application Support/Claude/claude_desktop_config.json
Backup:  ~/Library/Application
Support/Claude/claude_desktop_config.agentsgate.bak.json

Restart Claude Desktop to apply changes.
```

Claude Desktop を再起動してください。以降のすべてのツール呼び出しが AgentsGate を経由します。

連携の解除

```
agentsgate eject
```

6. データベース MCP サーバー（PostgreSQL / MySQL）連携

AgentsGate には PostgreSQL と MySQL に対応した組み込み MCP サーバーが付属しています。これらを使うと、AI エージェントがデータベースを操作する際もすべての SQL が AgentsGate を経由し、リスク評価・ログ記録・承認制御が適用されます。

6.1 PostgreSQL MCP サーバーの登録

Claude Desktop を閉じてから以下を実行してください：

```
agentsgate inject-pg --connection-string=postgresql://ユーザー:パスワード@ホスト:5432/データベース名
```

成功すると以下のような出力が表示されます：

```
[agentsgate] Registered "agentsgate-pg-database" in claude_desktop_config.json
connection: postgresql://ユーザー:***@ホスト:5432/データベース名
```

Restart Claude Desktop / Claude Code for the change to take effect.

Claude Desktop を再起動してください。以降、AI エージェントがデータベースを操作する際は AgentsGate がすべての SQL を監視します。

6.2 MySQL MCP サーバーの登録

Claude Desktop を閉じてから以下を実行してください：

```
agentsgate inject-mysql --connection-string=mysql://ユーザー:パスワード@ホスト:3306/データベース名
```

成功すると以下のような出力が表示されます：

```
[agentsgate] Registered "agentsgate-mysql-database" in claude_desktop_config.json
connection: mysql://ユーザー:***@ホスト:3306/データベース名
```

Restart Claude Desktop / Claude Code for the change to take effect.

Claude Desktop を再起動してください。

6.3 よく使うオプション

オプション	説明	例
--connection-string=<url>	接続先データベースの URL（必須）	postgresql://user:pass@localhost:5432/mydb
--name=<名前>	サーバーの登録名 （複数 DB を使う場合に区別するために使用）	--name=本番 DB
--force	既存の登録を上書きする	agentsgate inject-pg --connection-string=... --force
remove	登録したサーバーを	agentsgate inject-pg remove

	削除する	
--	------	--

6.4 登録の解除

PostgreSQL の登録を解除

agentsgate inject-pg remove

MySQL の登録を解除

agentsgate inject-mysql remove

6.5 データベース操作のリスクスコアについて

データベース MCP サーバーを通じた操作には、以下のリスクスコアが適用されます：

操作の種類	例	デフォルトスコア
テーブル一覧・スキーマ確認	list_tables, describe_table	0.05（許可）
SELECT クエリ	query（SELECT 文）	0.05（許可）
INSERT / UPDATE	execute（WHERE 句のある INSERT / UPDATE / DELETE ）	0.30（承認待ち）
DELETE / DROP / TRUNCATE	execute（WHERE 句なしの DELETE）, execute_ddl	0.90～1.00（ブロック）

⚠ DELETE や DROP は高リスクのためデフォルトでブロックされます。実行を許可したい場合はポリシーファイルを編集してください（docs/policy-guide.md 参照）。

7. ダッシュボードの使い方と安全な公開設定

⚠ ダッシュボードは既定で認証がありません

AgentsGate は既定で 127.0.0.1（ループバック）にのみ待ち受けます。同じ PC からしかアクセスできないため、既定では API キーが未設定でも第三者に読まれることはありません。

ただし操作ログには、AI エージェントが読み書きしたファイルの中身やデータベースの行、認証情報がそのまま記録されます。以下のいずれかに当てはまる場合は必ず API キーを設定してください。

- proxy.host を 127.0.0.1 以外に変更して外部から接続できるようにする場合
- 複数人が使う PC やサーバーで動かす場合

設定例（～/.agentsgate/config.json）：

```
{ "dashboard": { "apiKey": "32 文字以上のランダムな文字列" } }
```

設定すると、/health 以外のすべてのエンドポイントで X-API-Key ヘッダーが必要になります。

外部公開する場合は、認証付きのリバースプロキシを必ず前段に置いてください。プロキシ本体（ポート 4000）には認証機能がありません。

ブラウザを開き、以下の URL にアクセスします：

`http://localhost:4001`

主要タブの説明

タブ名	内容
Overview（概要）	操作のライブフィード、承認待ち、チェックポイント、リスク推移をまとめて表示
Agents（エージェント）	エージェントごとの操作数・平均リスク・ブロック率
Tools（ツール）	ツールごとの利用状況とリスク傾向
Rules（ルール）	ポリシールールの一覧・追加・削除・優先度変更
Quota（クォータ）	エージェントごとの 1 日あたり利用上限と消化状況
Circuit Breakers	連続失敗で遮断されたエージェントの状態と手動リセット

8. 実践シナリオ: PostgreSQL / MySQL でデータベース操作を保護する

このシナリオでは、実際にデータベースへ接続し、AI エージェントがデータベースを操作する際に AgentsGate がどのように動作するかを体験します。

所要時間：約 15 分 前提：PostgreSQL または MySQL のデータベースが用意されていること

8.1 シナリオの概要

以下の手順で AgentsGate によるデータベース保護を体験します：

- PostgreSQL（または MySQL）データベースに AgentsGate を接続する
- Claude にデータベースへの SQL 操作を依頼する
- ダッシュボードで各操作のリスクスコアと判定結果を確認する
- 高リスク操作（DROP TABLE など）が自動でブロックされることを確認する

💡 データベースは既存のものをそのまま使用できます。ローカルインストール・Docker・クラウドサービスのいずれでも動作します。

8.2 PostgreSQL に接続する

Claude Desktop を閉じてから以下を実行します：

```
agentsgate inject-pg --connection-string=postgresql://ユーザー:パスワード@localhost:5432/データベース名
```

成功すると以下のような出力が表示されます：

```
[agentsgate] Registered "agentsgate-pg-database" in claude_desktop_config.json
connection: postgresql://ユーザー:***@localhost:5432/データベース名
```

Restart Claude Desktop / Claude Code for the change to take effect.
Claude Desktop を再起動してください。

8.3 Claude にデータベース操作を依頼する

Claude Desktop を開き、以下の操作を順番に依頼します。

【テスト操作①】テーブル一覧の取得（低リスク）

Claude への指示例：「データベースにどんなテーブルがあるか教えてください。」

期待される動作：

- `list_tables` が呼び出され、AgentsGate がリスクスコア 0.05 で `allow` と判定する
- ダッシュボードの **Overview** タブに操作が記録される
- Claude がテーブル一覧を回答する

【テスト操作②】データの取得（低リスク）と書き込み（中リスク）

Claude への指示例：「`users` テーブルの最初の 5 件のレコードを見せてください。」（読み取りは許可されます。承認待ちを試すには「`users` テーブルの `status` を更新してください」のような書き込みを依頼してください — `execute` が 0.30 で `require_approval` になります）

期待される動作：

- `query` が呼び出され、AgentsGate がリスクスコア 0.05 で `allow` と判定する
- 書き込みを依頼した場合は **Overview** タブの承認待ち一覧に表示される
- 承認すると Claude が結果を表示する

【テスト操作③】テーブル削除の試行（高リスク・自動ブロック）

Claude への指示例：「`users` テーブルを削除してください。」

期待される動作：

- `execute_ddl` が呼び出され、AgentsGate がリスクスコア 1.00 で `block` と判定する
- Claude は「操作がブロックされました」というエラーを受け取る
- ダッシュボードに `block` として記録され、`firedRules` に `L1_DB_DROP` が表示される

8.4 ダッシュボードで操作を確認する

ブラウザで AgentsGate ダッシュボードを開きます：

<http://localhost:4001>

Overview タブの操作一覧でテスト操作①～③がすべて記録されていることを確認します。操作③の行をクリックし、以下を確認してください：

- decision: block
- riskScore: 1.00
- firedRules: ["L1_DB_DROP"]

CLI でも確認できます：

```
agentsgate ops tail --limit=10
agentsgate ops tail --action=block
```

8.5 MySQL に接続する（オプション）

MySQL を使う場合は inject-pg の代わりに inject-mysql を使用します：

```
agentsgate inject-mysql --connection-string=mysql://ユーザー:パスワード@localhost:3306/データベース名
```

その後の手順は PostgreSQL の場合と同じです。テスト操作①～③、ダッシュボードでの確認、クリーンアップまですべて同様に実施できます。

複数のデータベースを同時に AgentsGate で保護する場合は --name オプションで区別します：

```
agentsgate inject-pg --connection-string=postgresql://... --name=production-db
agentsgate inject-mysql --connection-string=mysql://... --name=staging-db
```

8.6 登録の解除とクリーンアップ

このシナリオが終わったら以下で登録を解除します：

```
# PostgreSQL の登録を解除
agentsgate inject-pg remove
```

```
# MySQL の登録を解除
agentsgate inject-mysql remove
```

⚠ データベース操作中に問題が起きた場合は、agentsgate checkpoints でチェックポイントを確認し、agentsgate rollback <checkpointId> で復元できます（snapshot_table パラメータを使った操作のみ対応）。

9. テスト実行とデバッグ情報の確認

AgentsGate には、エラーが発生したときのデバッグを容易にする仕組みが組み込まれています。このセクションでは、導入後のテスト手順と、問題が起きたときのデバッグ情報の確認方法を説明します。

9.1 動作テスト — 基本チェックリスト

AgentsGate を起動した後、以下の順序で動作確認を行ってください。

13. **ステップ①** agentsgate start を実行し、プロキシとダッシュボードが起動するか確認する

14. **ステップ②** 別ターミナルで `agentsgate status` を実行し、「running」と表示されるか確認する
15. **ステップ③** ブラウザで `http://localhost:4001` を開き、ダッシュボードが表示されるか確認する
16. **ステップ④** `agentsgate health` を実行し、DB・プロキシの健全性を確認する
17. **ステップ⑤** Claude Desktop（または任意の MCP クライアント）から簡単な操作を実行し、ダッシュボードに記録されるか確認する

health コマンドの出力例：

```
$ agentsgate health
AgentsGate OK - v0.1.0

Uptime:          12m 34s (since 2026-07-29T09:02:11.418Z)
Operations:      42
Pending approvals: 0

DB row counts:
operation_logs:  42
checkpoints:     3
pending_approvals: 0
```

9.2 デバッグモード（AGENTSGATE_DEBUG=1）

問題が発生したとき、またはプロキシ内部で何が起きているかを詳しく知りたいときは、デバッグモードを有効にして起動します。

デバッグモードでは、エラーが発生するたびに以下の情報が `stderr`（ターミナル）に即時出力されます：

- エラーが発生したモジュール名（`proxy`, `checkpoint`, `dashboard` など）
- エラーメッセージ
- スタックトレース（どのファイルの何行目で発生したか）
- 関連する `operationId`（操作 ID）
- 追加コンテキスト情報

起動コマンド（Windows コマンドプロンプト）：

```
set AGENTSGATE_DEBUG=1 && agentsgate start
```

起動コマンド（macOS / Linux / Git Bash）：

```
AGENTSGATE_DEBUG=1 agentsgate start
```

デバッグモード時の出力例：

```
[AgentsGate] ERROR [proxy] Risk engine timeout
Error: Risk engine timeout
    at RiskScoringEngine.assess (src/modules/m6-risk/index.ts:87:13)
```

```
at evaluateRisk (src/modules/ml-proxy/index.ts:192:24)
at MCPProxy.intercept (src/modules/ml-proxy/index.ts:241:18)
operationId: 3fa85f64-5717-4562-b3fc-2c963f66afa6
context: {"tool":"file_system","method":"write"}
```

💡 ポイント：デバッグモードは開発・トラブルシューティング時のみ使用してください。通常運用では AGENTSGATE_DEBUG=1 を外して起動することを推奨します。

9.3 エラー履歴の確認（agentsgate errors コマンド）

AgentsGate は直近 200 件のエラーをメモリ内に保持しています。起動中にいつでも以下のコマンドで確認できます。

```
# 直近 10 件のエラーを表示
agentsgate errors 10

# デフォルト (50 件)
agentsgate errors
```

出力例：

TIMESTAMP	MODULE	MESSAGE
2026-03-22T10:05:31.123Z	proxy	Risk engine timeout
2026-03-22T10:03:14.456Z	proxy	Connection refused: 127.0.0.1:9999

エラーが 0 件の場合：

```
No errors recorded.
```

9.4 REST API でエラーを確認する（上級者向け）

ダッシュボード API から JSON 形式でエラー一覧を取得することもできます：

```
# 直近 20 件
curl http://localhost:4001/errors?limit=20

# API キーが設定されている場合
curl -H "X-API-Key: <your-key>" http://localhost:4001/errors
```

レスポンス例：

```
{
  "errors": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "timestamp": "2026-03-22T10:05:31.123Z",
      "module": "proxy",
      "message": "Risk engine timeout",
      "stack": "Error: Risk engine timeout\n    at ...",
      "operationId": "op-abc123"
    }
  ],
  "total": 1
}
```

9.5 操作ログで問題の原因を追跡する

エラーが発生した操作の詳細は操作ログから追跡できます：

```
# 最近の操作を表示
agentsgate ops tail --limit=20

# 特定の操作 ID の詳細を表示
agentsgate ops get <operationId>

# ブロックされた操作だけ表示
agentsgate ops tail --action=block
```

9.6 デバッグ情報の読み方

問題が発生したとき、以下の順序で確認することを推奨します：

18. **1. エラー履歴を確認** agentsgate errors で何のエラーが発生しているか確認する
19. **2. 操作ログを確認** エラー内の operationId を使い agentsgate ops get <id> で操作詳細を見る
20. **3. デバッグモードで再現** AGENTSGATE_DEBUG=1 で起動し、スタックトレースを確認する
21. **4. ヘルス状態を確認** agentsgate health で DB・プロキシに異常がないか確認する
22. **5. doctor コマンドで自己診断** agentsgate doctor でシステム全体の健全性チェックを実行する

doctor コマンドの出力例：

```
$ agentsgate doctor

AgentsGate Doctor

✓ Config file          port=4000 dashboard=4001
✓ Policy file          0 rule(s) loaded
✓ State directory      ~/.agentsgate
✓ SQLite database      ~/.agentsgate/agentsgate.db (accessible, empty)
✓ Shadow git repo      ~/.agentsgate/shadow-repo
✓ Proxy process        running (PID 43427)
✓ Claude Desktop config 1 server(s), 1 injected

All checks passed.
```

9.7 操作ログが改ざんされていないか確認する

config.json に audit.signingSecret を設定しておく、と、記録される操作ログ 1 件ごとに署名が付きます。各署名は直前の記録の署名を含んでいるため、記録が書き換えられた場合だけでなく、都合の悪い記録を削除された場合も検出できます。

```
{"audit": {"signingSecret": "長いランダムな文字列"}}
```

確認するには：

agentsgate verify-logs

Chain: intact と表示されれば、記録の書き換えも削除も行われていません。破断が見つかった場合は何件目で途切れたかが表示されます。

⚠ 最新の記録がまとめて削除された場合は検出できません（短いが整合した状態になるため）。厳密に必要な場合は、最新の署名を別の場所に控えてください。

10. よく使うコマンド一覧

コマンド	説明
agentsgate start	プロキシとダッシュボードを起動（バックグラウンド）
agentsgate start 8080	ポート 8080 で起動（ダッシュボードは 8081）
agentsgate start --foreground	前面で起動（Ctrl+C で停止）
agentsgate stop	起動中のプロキシを停止
agentsgate status	現在の起動状態とポートを表示
agentsgate health	ダッシュボードの健全性確認
agentsgate doctor	システム全体の自己診断
agentsgate config	現在の有効な設定を表示
agentsgate inject	Claude Desktop を自動設定
agentsgate eject	Claude Desktop の設定を解除
agentsgate inject-pg --connection-string=<url>	PostgreSQL MCP サーバーを登録
agentsgate inject-mysql --connection-string=<url>	MySQL MCP サーバーを登録
agentsgate inject-db --db=<path>	SQLite MCP サーバーを登録
agentsgate inject-pg inject-mysql inject-db remove	登録を解除
agentsgate ops tail	最近の操作を一覧表示
agentsgate ops get <id>	特定の操作の詳細を表示
agentsgate ops summary	操作の集計サマリー
agentsgate errors 20	直近のエラーを 20 件表示
agentsgate audit	監査ログを表示
agentsgate verify-logs	操作ログの署名とチェーンを検証
agentsgate checkpoints	チェックポイント一覧を表示
agentsgate snapshot list	スナップショットを一覧表示
agentsgate rollback <id>	指定チェックポイントに復元
agentsgate report	リスクサマリーレポートを生成

11. よくある質問・トラブルシューティング

「コマンドが見つかりません: agentsgate」

npm のグローバル bin ディレクトリが PATH に含まれていない可能性があります。

npm bin -g で表示されるパスを PATH に追加してください。

または `npx agentsgate` を代わりに使用してください。

「ポートが使用中」エラー

ポート 4000 または 4001 が他のアプリで使われています。

```
agentsgate start 4100
```

で別のポートを指定して起動してください。

Claude Desktop が接続しない

1. Claude Desktop を閉じてから `agentsgate inject` を実行したか確認してください。
2. inject 後に Claude Desktop を再起動したか確認してください。
3. `agentsgate status` でプロキシが起動中か確認してください。

エラーが出たがメッセージが分からない

`AGENTSGATE_DEBUG=1` を付けて起動すると、スタックトレースが表示されます。

また `agentsgate errors` でエラー履歴を確認してください。

操作がすべてブロックされる

ポリシーの介入しきい値 (`intervention.blockAtOrAbove`) が低すぎる可能性があります。

`agentsgate config` でしきい値を確認し、

必要に応じて `config.json` の `intervention.blockAtOrAbove` を調整してください。

ダッシュボードが開かない

`agentsgate start` が正常に起動しているか確認してください。

ファイアウォールがポート 4001 をブロックしていないか確認してください。

`agentsgate health` を実行してエラーがないか確認してください。

データベース MCP サーバーに接続できない

- 接続文字列 (`--connection-string`) のホスト・ポート・ユーザー名・パスワードが正しいか確認してください。
- データベースサーバーが起動中かどうか確認してください (`pg: pg_isready`、`mysql: mysqladmin ping`)。
- `agentsgate inject-pg remove` を実行してから `agentsgate inject-pg --connection-string=<url>` で再登録してください。
- `AGENTSGATE_DEBUG=1` で起動し、接続エラーの詳細を確認してください。

PostgreSQL / MySQL の操作がすべてブロックされる

- DELETE ・ DROP ・ TRUNCATE はデフォルトでブロックされます。これは正常な動作です。
- SELECT がブロックされる場合は `intervention.blockAtOrAbove` が低すぎる可能性があります。`agentsgate config` でしきい値を確認してください。
- 操作を許可したい場合は `policy.json` にルールを追加してください (`docs/policy-guide.md` 参照)。

12. アンインストール

以下の順序でアンインストールしてください：

1. データベース MCP サーバーの登録を解除する（登録している場合）

```
agentsgate inject-pg remove  
agentsgate inject-mysql remove
```

2. Claude Desktop の連携を解除

```
agentsgate eject
```

3. AgentsGate を停止

```
agentsgate stop
```

4. パッケージをアンインストール

```
npm uninstall -g agentsgate
```

次のステップ

- カスタムセキュリティルールの作成 → `docs/policy-guide.md`
- REST API の利用方法 → `docs/api-reference.md`
- セキュリティのベストプラクティス → `SECURITY.md`
- OpenClaw との連携 → `docs/openclaw-user-guide.md`